NLTK

```
import nltk
import pandas as pd
import re
from nltk.corpus import movie_reviews, stopwords
from nltk.stem import WordNetLemmatizer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, accuracy_score

# === 1. Download Required Resources === #
nltk.download('movie_reviews')
nltk.download('stopwords')
nltk.download('wordnet')

# === 2. Load Movie Reviews from nltk === #
documents = [
    (' '.join(movie_reviews.words(fileid)), category)
    for category in movie_reviews.categories()
    for fileid in movie_reviews.fileids(category)
]

df = pd.DataFrame(documents, columns=['review', 'label'])

# === 3. Display Some Records === #
print("□ Sample Movie Reviews:\n")
print(df.head(5))  # Show first 5 records
print("\nLabel Distribution:\n", df['label'].value_counts())
```

```
[nltk_data] Downloading package movie_reviews to
[nltk_data]     C:\Users\Madhu\AppData\Roaming\nltk_data...
[nltk_data]   Package movie_reviews is already up-to-date!
[nltk_data] Downloading package stopwords to
[nltk_data]     C:\Users\Madhu\AppData\Roaming\nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to
[nltk_data]     C:\Users\Madhu\AppData\Roaming\nltk_data...
[nltk_data]   Package wordnet is already up-to-date!

□ Sample Movie Reviews:

                                              review label
0  plot : two teen couples go to a church party ,...   neg
1  the happy bastard ' s quick movie review damn ...   neg
2  it is movies like these that make a jaded movi...   neg
3  " quest for camelot " is warner bros . ' first...   neg
```

```
4  synopsis : a mentally unstable man undergoing ...   neg

Label Distribution:
 label
neg    1000
pos    1000
Name: count, dtype: int64
```

```
# === 4. Text Cleaning === #
stop_words = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()

def clean_text(text):
    text = text.lower()
    text = re.sub(r'[^a-z\s]', '', text)
    tokens = text.split()
    tokens = [lemmatizer.lemmatize(word) for word in tokens if word
not in stop_words]
    return ' '.join(tokens)

df['cleaned_review'] = df['review'].apply(clean_text)

# === 5. TF-IDF Vectorization === #
vectorizer = TfidfVectorizer(max_features=5000)
X = vectorizer.fit_transform(df['cleaned_review'])
y = df['label']

# === 6. Train/Test Split === #
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# === 7. Train Classifier === #
clf = LogisticRegression()
clf.fit(X_train, y_train)

# === 8. Evaluate Model === #
y_pred = clf.predict(X_test)
print("\n Accuracy:", accuracy_score(y_test, y_pred))
print("\n Classification Report:\n", classification_report(y_test,
y_pred))
```

```

 Accuracy: 0.8225

 Classification Report:
              precision    recall  f1-score   support

         neg       0.82      0.82      0.82       199
         pos       0.82      0.82      0.82       201

    accuracy                           0.82       400
```

```
   macro avg       0.82      0.82      0.82       400
weighted avg       0.82      0.82      0.82       400
```

Local Dataset

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, accuracy_score

# Load dataset
df = pd.read_csv('movie_review.csv')
print(df.head())
```

```
   fold_id cv_tag  html_id  sent_id  \
0        0  cv000    29590        0
1        0  cv000    29590        1
2        0  cv000    29590        2
3        0  cv000    29590        3
4        0  cv000    29590        4

                                                text  tag
0  films adapted from comic books have had plenty...  pos
1  for starters , it was created by alan moore ( ...  pos
2  to say moore and campbell thoroughly researche...  pos
3  the book ( or " graphic novel , " if you will ...  pos
4  in other words , don't dismiss this film becau...  pos
```

```
import re
import string
from sklearn.feature_extraction.text import TfidfVectorizer
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
import nltk

# Ensure the required columns exist
assert 'text' in df.columns and 'tag' in df.columns,

# Optional: check unique labels
print("Unique Labels:", df['tag'].unique())
```

```
Unique Labels: ['pos' 'neg']
```

```
nltk.download('stopwords')
nltk.download('wordnet')
```

```
[nltk_data] Downloading package stopwords to
[nltk_data]     C:\Users\Madhu\AppData\Roaming\nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to
[nltk_data]     C:\Users\Madhu\AppData\Roaming\nltk_data...
[nltk_data]   Package wordnet is already up-to-date!

True
```

```
# === Text Preprocessing === #
stop_words = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()

def clean_text(text):
    text = text.lower()  # lowercase
    text = re.sub(r'<[^>]+>', '', text)  # remove HTML tags
    text = re.sub(r'\d+', '', text)  # remove numbers
    text = re.sub(r'[' + string.punctuation + ']', '', text)  # remove
punctuation
    tokens = text.split()
    tokens = [lemmatizer.lemmatize(w) for w in tokens if w not in
stop_words]
    return ' '.join(tokens)

# Apply cleaning
df['cleaned_review'] = df['text'].astype(str).apply(clean_text)

# === Feature Extraction (TF-IDF) === #
vectorizer = TfidfVectorizer(max_features=5000)
X = vectorizer.fit_transform(df['cleaned_review'])

# Convert labels to binary (0 = neg, 1 = pos)
y = df['tag'].apply(lambda x: 1 if x == 'pos' else 0)

# === Train-Test Split === #
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# === Model Training (Logistic Regression) === #
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

# === Prediction & Evaluation === #
y_pred = model.predict(X_test)
print("\nAccuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test,
y_pred))
```

```
Accuracy: 0.6688813349814586
```

```
Classification Report:
              precision    recall  f1-score   support

           0       0.67      0.66      0.66      6371
           1       0.67      0.68      0.68      6573

    accuracy                           0.67     12944
   macro avg       0.67      0.67      0.67     12944
weighted avg       0.67      0.67      0.67     12944
```